

Partition to K Equal Sum Subsets

Algorithm Analysis & Symmetry Pruning Optimization (LeetCode 698)

Authored by: Arpit Raj, LNMIIT Jaipur

Date: July 28, 2026

Part 1: Code Breakdown & Approach

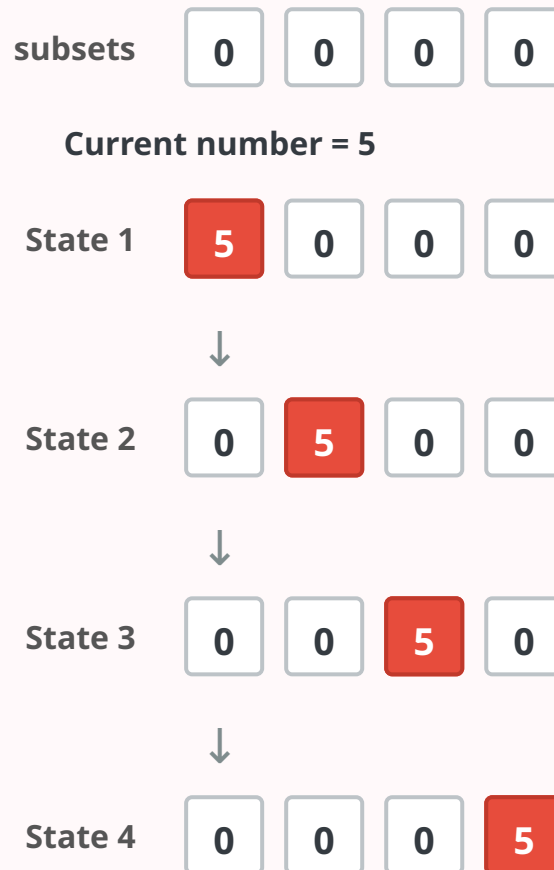
This problem requires dividing an array of numbers into `k` subsets such that the sum of elements in each subset is identical. We tackle this using Depth-First Search (DFS) with Backtracking.

Algorithm Execution Steps:

- **Initial Checks:** First, we calculate the total sum of all elements. If `sum % k != 0`, it's mathematically impossible to partition them equally, so we return false. Otherwise, we define our target sum for each subset as `target = sum / k`.
- **Sorting Optimization:** The array is sorted in descending order (`greater<int>()`). Trying to fit larger elements into subsets first is a classic optimization constraint—it forces failures early in the recursion tree, pruning dead ends quickly.
- **Backtracking Core:** We use an array `subsets` of size `k` (all initialized to 0). The `dfs` function attempts to place the element at `nums[idx]` into one of the available subsets `subsets[i]`.
- **State Progression:** If adding the element does not exceed the `target`, we add it and recurse for the next element (`idx + 1`). If it eventually returns true, we are done. Otherwise, we backtrack by removing the element and try the next subset.
- **Base Case:** When `idx == n`, we have successfully placed all elements, returning true.

Part 2: Missing Optimization (Symmetry Pruning)

Suppose:



These are all **identical states**. We are exploring the same thing k times.

The Fix: Write

```
if(subsets[i] == 0) {  
    break;  
}
```

Why?

Suppose:

Bucket 1 = 0

Bucket 2 = 0

Bucket 3 = 0

If putting Aadya's current stick into bucket 1 didn't work, putting it into bucket 2 will produce exactly the same search tree. No need to try. **This single line cuts huge amounts of recursion.**

Source Code

Complete C++ Implementation

```
class Solution {
public:

    int n;
    int target;

    bool dfs(vector<int>& nums, int idx, vector<int>& subsets){

        if(idx==n) return true; //saare daal diye
        int k=subsets.size();
        for(int i=0;i<k;i++){

            if(nums[idx]+subsets[i]>target) continue;
            subsets[i]+=nums[idx];

            if(dfs(nums,idx+1,subsets)){
                return true;
            }

            subsets[i]-=nums[idx];
            if(subsets[i]==0) break; //very importaant optimization line
                                   //symmetry pruning
        }
        return false;
    }

    bool canPartitionKSubsets(vector<int>& nums, int k) {

        //using the same approach as lc473 , ek baar uska solution dekh ke aao pehle mera
        n=nums.size();

        int sum=accumulate(nums.begin(),nums.end(),0);
        if(sum%k!=0) return false;

        target=sum/k;
        sort(nums.begin(),nums.end(),greater<int>());
        //pehle bade numbers fit krne ki try krenge

        vector<int> subsets(k,0);
        //initially kisi ko kuch ni mila

        return dfs(nums,0,subsets); //starting from 1st element
    }
};
```